Entendendo o artigo CheckList para apresentar

> **Conteúdo gerado por IA (Claude).** Escrito a partir da leitura do PDF completo do artigo. Revise antes de apresentar.

Artigo: Ribeiro, M. T.; Wu, T.; Guestrin, C.; Singh, S. *Beyond Accuracy: Behavioral Testing of NLP Models with CheckList*. ACL 2020, p. 4902–4912. **PDF:** https://aclanthology.org/2020.acl-main.442.pdf (11 páginas) **Slides do seminário:** https://claude.ai/artifact/H2kUtV9vqwkdLNNRsoDxKc

---

A ideia em 30 segundos

Modelos de linguagem costumam ser avaliados por **um número**: a acurácia num conjunto de teste. Um modelo com 92% parece ótimo. O artigo mostra que esse número esconde falhas graves. O mesmo modelo pode errar quase sempre frases simples como *"I thought the plane would be awful, but it wasn't"* ("achei que o avião seria horrível, mas não foi").

A proposta, chamada **CheckList**, é testar modelos de linguagem **como se testa software**: uma habilidade de cada vez, com muitos casos de teste gerados automaticamente.

A frase para guardar: **o CheckList troca a pergunta "quanto o modelo acerta?" por "o que o modelo realmente sabe fazer?"**

Mapa dos slides

A apresentação tem 14 slides: capa, 4 partes de 3 slides cada e referências.

Parte	Slides	Assunto
—	1	Capa
Parte 1	2 · 3 · 4	Motivação · O que o artigo contribui · Trabalhos relacionados
Parte 2	5 · 6 · 7	Acurácia × teste comportamental · Capacidade, perturbação e teste metamórfico · Tarefas e modelos
Parte 3	8 · 9 · 10	A matriz · Os três tipos de teste · Gerando testes e desenho experimental
Parte 4	11 · 12 · 13	Resultados principais · Aplicações do CheckList · Conclusão
—	14	Referências

Parte 1 — Motivação, contribuições e trabalhos relacionados

O problema que motivou o artigo (slide 2)

A forma padrão de avaliar um modelo de PLN (Processamento de Linguagem Natural) é:

1. separar os dados em treino, validação e teste;
2. treinar o modelo;
3. medir a acurácia no conjunto de teste, muitas vezes num ranking público (*leaderboard*).

Os autores apontam três defeitos nisso:

- **O conjunto de teste tem os mesmos vieses do treino.** Por isso o desempenho no mundo real pode ser bem pior que o medido.
- **Um número só não diz onde o modelo erra** nem como corrigir.
- **As alternativas que existiam eram fragmentadas:** cada uma olhava uma tarefa ou uma habilidade (só robustez, só justiça…). Faltava um guia geral do que testar.

O exemplo do slide: o BERT tem **92,7%** de acurácia no benchmark de sentimento SST-2, mas erra **85%** das frases simples com a negação no fim, como *“I thought the plane would be awful, but it wasn’t”*. A resposta dos autores: testar modelos de linguagem como se testa software.

Como fechar o slide: “Então, se a acurácia não basta, o que o artigo propõe?” e passar para as contribuições.

As contribuições (slide 3)

O slide resume o artigo em **três contribuições**:

#	Contribuição	Em palavras simples
1	**Um método**	Testar modelos de linguagem como se testa software: **uma habilidade de cada vez** (negação, nomes…), em vez de uma nota só. Serve para qualquer tarefa e qualquer modelo.
2	**Uma ferramenta gratuita**	Gera **milhares de testes automaticamente**, a partir de modelos de frase. Disponível em github.com/marcotcr/checklist.
3	**Evidências**	Achou bugs graves em modelos de pesquisa (BERT) e em produtos da Microsoft, do Google e da Amazon. Quem usou o CheckList achou **cerca de 3× mais bugs**.

Os detalhes ficam para depois: a lista de habilidades e os tipos de teste aparecem na Parte 3; os números dos bugs, na Parte 4.

Como resumir na fala: “Three things: a method, a tool, and evidence that it works.”

Trabalhos relacionados (slide 4)

O título do slide é **“Before CheckList: one piece at a time”** (“antes do CheckList: um pedaço de cada vez”). A mensagem: **cada abordagem anterior resolvia só um pedaço do problema.**

Cartão	O que era	Por que não bastava
Accuracy	A nota geral num benchmark	É **um número só**: não mostra onde o modelo erra
Robustness and fairness tests	Testes com erros de digitação, troca de nomes…	Olham **uma habilidade ou uma tarefa** de cada vez
Challenge sets	Conjuntos de frases difíceis	São **pequenos e feitos à mão**
Probes (sondas)	Olham o interior do modelo	Não mostram como ele **se comporta** na tarefa real

A frase de baixo: o CheckList junta tudo isso num método só, usando ideias do **teste de software** (teste unitário e teste metamórfico).

Exemplo bom para as sondas: uma sonda mostra que o BERT “sabe” reconhecer nomes (96,7%), mas o BERT usado numa tarefa real erra testes simples com nomes. Saber algo por

dentro não garante usar isso na prática.

O que o CheckList não cobre, segundo os autores: erros de rótulo, versão dos dados, ataques de segurança e interpretabilidade. Não está no slide, mas é bom saber se perguntarem.

Parte 2 — Conceitos que você precisa entender

Acurácia × teste comportamental (slide 5)

- **Acurácia em dados reservados (*held-out accuracy*):** porcentagem de acertos em exemplos que o modelo não viu no treino. Exemplo: o BERT tem 92,7% no SST-2 (resenhas de filmes).
- **Teste comportamental (caixa-preta):** testar só pela entrada e saída, sem olhar código ou pesos. Cada teste verifica um comportamento.

Analogia: a acurácia é a **média final** do aluno. O teste comportamental é o **boletim por matéria**. Um aluno com média 9 pode estar tirando zero em uma matéria.

Capacidade, perturbação e teste metamórfico (slide 6)

Conceito	O que é	Exemplo
Capacidade	Uma habilidade que a tarefa exige	Negação: “not poor” (não é ruim) não é negativo
Perturbação	Uma mudança controlada na entrada	Trocar “Chicago” por “Dallas” num tweet
Teste metamórfico	Compara saídas entre si, sem precisar do gabarito	Se só troquei a cidade, o sentimento tem que continuar igual

Por que o teste metamórfico é importante: eu não sei qual é o sentimento “certo” de um tweet qualquer, mas sei que trocar o nome da cidade não deveria mudá-lo. Então dá para testar com dados reais **sem rótulo**.

Analogia: é como conferir uma balança. Sem saber seu peso exato, você sabe que tirar o relógio não pode fazer o número subir.

As tarefas e os modelos (slide 7)

Tarefa	O que o modelo precisa fazer
Análise de sentimento	Dizer se um texto é positivo, negativo ou neutro
Pares de perguntas do Quora (QQP)	Dizer se duas perguntas perguntam a mesma coisa
Compreensão de leitura (SQuAD)	Responder a uma pergunta a partir de um parágrafo

- **BERT e RoBERTa:** modelos de linguagem do tipo Transformer, pré-treinados com muito texto e depois ajustados (*fine-tuned*) para cada tarefa.
- **Modelo de linguagem mascarado:** adivinha a palavra escondida numa frase. Com "I really [mask] the flight", sugere *enjoyed*, *liked*, *hated*... O CheckList usa isso para ajudar a gerar testes.

Parte 3 — Metodologia

A matriz (slide 8)

O teste é organizado numa tabela:

- **linhas** = capacidades (vocabulário, nomes próprios, negação...);
- **colunas** = tipos de teste (MFT, INV, DIR);
- **cada célula** = a taxa de falha, ou seja, a porcentagem de casos que o modelo errou.

Exemplo do artigo (Figura 1), com um modelo comercial de sentimento:

Capacidade	MFT	INV	DIR
Vocabulário	15,0%	16,2%	34,6%
Nomes próprios	0,0%	20,8%	—
Negação	**76,4%**	—	—

Ou seja: o modelo erra 76,4% de um teste simples de negação.

Os três tipos de teste (slide 9)

Tipo	Ideia	Exemplo	O que deve acontecer
MFT – teste de funcionalidade mínima	Casos simples com resposta certa (como um teste unitário)	"I didn't love the flight."	Negativo
INV – teste de invariância	Uma mudança que **não** deveria importar	Trocar "Cuba" por "Canada"	Mesma previsão
DIR – teste de expectativa direcional	Uma mudança com efeito conhecido	Acrescentar "You are lame."	O sentimento não pode ficar mais positivo

Ponto-chave: INV e DIR **não precisam de rótulo**. Comparam a resposta antes e depois da mudança.

Exemplo do dia a dia (um app que lê avaliações de restaurante):

- **MFT:** "A comida não estava ruim" tem que sair positivo ou neutro, nunca negativo.
- **INV:** "restaurante em Curitiba" → "restaurante em Recife" não pode mudar a nota.
- **DIR:** acrescentar "Péssimo atendimento" não pode deixar a avaliação melhor.

Gerando testes e desenho experimental (slide 10)

Este slide junta duas coisas: como os testes são gerados (a faixa escura no topo) e como o estudo foi organizado (os dois cartões A e B embaixo).

Gerando muitos testes

Escrever milhares de testes à mão é inviável. A ferramenta usa **modelos de frase** (*templates*):

```
I {NEGATION} {POS_VERB} the {THING}.
{NEGATION} = didn't, can't say I, ...
{POS_VERB} = love, like, ...
{THING}    = food, flight, service, ...
```

Cada combinação vira um caso de teste: com listas pequenas, já são centenas de frases.

A ferramenta também:

- usa o **RoBERTa para sugerir palavras** para as listas;
- traz **listas prontas** de nomes de pessoas, cidades, países e grupos;
- traz **perturbações prontas**, como trocar letras vizinhas para simular erro de digitação;
- permite **salvar e compartilhar** as suítes de teste.

Analogia: o modelo de frase é uma fábrica. “Eu {não gostei / não recomendo} do {comida / atendimento / preço}” gera 2 × 3 = 6 frases de uma vez.

Desenho experimental

Parte A — testar os modelos

Tarefa	Modelos testados	Desempenho no benchmark
Sentimento	APIs pagas da Microsoft, do Google e da Amazon	não informado
Sentimento	BERT e RoBERTa ajustados no SST-2	92,7% e 94,8%
Pares do Quora	BERT e RoBERTa ajustados no QQP	91,1% e 91,3%
Compreensão de leitura	BERT-large ajustado no SQuAD	93,2 de F1

- Para sentimento, os testes INV e DIR usaram **tweets sobre companhias aéreas, sem rótulo**.
- **Quando um teste falha:**
  - no MFT, quando a resposta é diferente da esperada;
  - no INV e no DIR, quando a previsão muda (ou vai na direção proibida) **mais de 0,1** em probabilidade. Isso evita contar oscilações mínimas.

Parte B — testar com pessoas

1. **Equipe da Microsoft:** sessão de cerca de 5 horas com a equipe que mantém o modelo comercial de sentimento. Os autores apresentaram o método **sem mostrar os testes deles**. A equipe pensou em uns 30 testes e implementou uns 20.
2. **Experimento controlado:** 18 pessoas com experiência intermediária em PLN (8 da indústria, 10 da academia), 2 horas para testar o BERT no Quora, divididas em três grupos de 6:
    - **sem ajuda;**
    - **só com a lista de capacidades;**
    - **com a ferramenta completa.**

    Cada pessoa deu uma nota de **gravidade** aos bugs, de 1 ("não é bug") a 5 ("nenhum modelo com esse bug deveria estar em produção").

Parte 4 — Resultados, aplicações e conclusão

Resultados principais (slide 11)

Este slide tem duas colunas: à esquerda, as falhas dos modelos; à direita, os resultados com usuários.

Os modelos falham no básico (coluna da esquerda)

Número	O que aconteceu
100%	Negação no fim da frase ("…would be awful, but it wasn't"): Microsoft e Amazon erram todos os casos
100%	Compreensão de leitura: erra todos os testes simples de "ele"/"ela" ("He is a journalist, and she is an adviser. Who is a journalist?")
98%	Pares do Quora: o BERT confunde "before" (antes) com "after" (depois)
89%	"John is not a doctor, Mary is. Who is a doctor?" → o modelo responde John. Com os nomes trocados, erra só 3,2%: **viés de gênero**

Outras descobertas importantes:

- **Os modelos de pesquisa foram melhores que os comerciais** na maioria dos testes, o que surpreendeu os autores. A exceção são as frases neutras: o BERT e o RoBERTa erram quase todas porque foram treinados só com positivo e negativo.
- **A acurácia escondia o problema:** o BERT acerta mais de 90% das frases com negação do próprio benchmark SST-2, mas falha nos testes simples de negação do CheckList.
- **Interpretação dos autores:** os modelos usam **atalhos**, por exemplo se apoiar nos nomes próprios, em vez de entender de verdade.

Quem usou o CheckList achou mais bugs (coluna da direita)

Comparando o grupo com a ferramenta completa e o grupo sem ajuda:

Medida	Sem ajuda	Com CheckList	Diferença
Testes criados	5,8	13,5	**2×**
Casos por teste	7,3	198	**27×**
Bugs graves (gravidade ≥ 3)	2,2	6,2	**≈3×**

- Uma pessoa de fora reavaliou a gravidade dos bugs e chegou a resultados quase iguais.
- A **equipe da Microsoft** achou bugs novos num modelo maduro. Também testou coisas que não tinha pensado, como hashtags (“#IHateYou”) e negação implícita (“I wish it was good”), e disse que vai adotar o CheckList.
- **Ressalva:** cada grupo tinha só 6 pessoas e o artigo não traz teste estatístico. É uma indicação forte, não uma prova.

Aplicações do CheckList (slide 12)

O slide mostra **onde o CheckList pode ser usado na prática**, segundo o próprio artigo. São quatro aplicações:

Aplicação	O que significa	De onde vem no artigo
Escolher um modelo	Comparar modelos habilidade por habilidade, e não por uma nota só	Os autores compararam modelos de pesquisa com os serviços da Microsoft, do Google e da Amazon
Ciclo de desenvolvimento	Rodar a mesma bateria de testes a cada nova versão do modelo	A equipe da Microsoft disse que vai incorporar o CheckList ao ciclo de desenvolvimento e corrigir os bugs na próxima versão
Benchmarks e pipelines de avaliação	Acrescentar os testes aos benchmarks e às avaliações que já existem	Conclusão do artigo: os testes servem para qualquer modelo e entram facilmente em benchmarks e pipelines
Suítes de teste compartilhadas	Equipes criam e reaproveitam testes juntas	Os autores esperam criação colaborativa, porque muitos testes servem para várias tarefas (erros de digitação, troca de nomes)

A frase embaixo do slide: tudo isso funciona com qualquer modelo, até uma API paga, porque o CheckList é caixa-preta: não precisa dos dados de treino nem dos pesos do modelo.

O que falar: “Where can CheckList be used in practice? To choose between models, to re-test every new version, to extend existing benchmarks, and to build shared test suites. And it works on any model, even a paid API, because it only needs inputs and outputs.”

Conclusão (slide 13)

- **A acurácia é útil, mas não basta.**
- **Pontos fortes:** o CheckList serve para qualquer tarefa e modelo, acha bugs até em produtos comerciais e é fácil de aprender.
- **Limitações:**
  - testa só comportamento (não pega erro de rótulo nem problema de segurança);
  - **passar num teste não prova que o modelo está certo** (os comerciais passaram no teste simples de justiça, e isso não prova que são justos);
  - o estudo com usuários é pequeno (essa é uma observação nossa, não dos autores).
- **Trabalho futuro proposto pelos autores:** criar suítes de teste em colaboração, já que muitos testes servem para várias tarefas.

Analogias para usar na apresentação

- **Boletim escolar:** a acurácia é a média; o CheckList é a nota por matéria.
- **Inspeção de carro:** ninguém aprova um carro só porque ele andou 100 km; existe uma lista (freio, farol, pneu). A matriz é essa lista para modelos de linguagem.
- **Aluno que decorou palavras-chave:** acerta a prova de sempre, erra quando um detalhe muda (“before” por “after”). É o que os autores chamam de atalho.
- **Balança:** sem saber seu peso exato, você sabe que tirar o relógio não pode aumentar o número. É assim que INV e DIR funcionam sem gabarito.

Perguntas que podem aparecer (com respostas curtas)

1. **Qual o problema da acurácia?** Superestima o desempenho real, tem os vieses do treino e não diz onde o modelo erra.
2. **O que é teste comportamental?** Testar só pela entrada e saída, sem olhar o interior do modelo. Por isso dá para testar APIs comerciais.
3. **Qual a diferença entre MFT, INV e DIR?** MFT: caso simples com resposta certa. INV: mudança que não deve alterar a resposta. DIR: mudança com efeito conhecido na resposta.
4. **Como testar sem rótulo?** INV e DIR comparam a resposta com a resposta anterior do próprio modelo, não com um gabarito.
5. **Quais foram as falhas mais graves?** Negação (perto de 100%), correferência (100%), noção de tempo (“before”/“after”) e papéis semânticos (quem fez o quê).
6. **Por que os modelos de pesquisa foram melhores que os comerciais?** O artigo trata isso como surpresa e não dá uma explicação fechada.
7. **Os modelos comerciais são justos, já que passaram no teste de justiça?** Não dá para concluir isso. Passar num teste simples não prova ausência de viés.
8. **A gravidade dos bugs não é subjetiva?** É. Para reduzir isso, uma pessoa de fora reavaliou e o resultado foi quase igual. Mas com 6 pessoas por grupo é uma indicação, não prova.
9. **Em que o CheckList difere das sondas?** Sondas olham o interior do modelo; o CheckList testa o comportamento na tarefa final.

10. **Quais as limitações?** Não cobre erros de rótulo, versão dos dados, ataques nem interpretabilidade. E depende da criatividade de quem escreve os testes.
11. **Onde o CheckList pode ser usado na prática?** Para escolher entre modelos, para testar de novo cada nova versão, para ampliar benchmarks e pipelines de avaliação e para criar suítes de teste compartilhadas entre equipes.

Antes de apresentar, confira no PDF

- O artigo ganhou o prêmio de melhor artigo da ACL 2020. Essa informação não está no texto do PDF; confirme antes de citar.
- Qual modelo previu sempre negativo no teste de justiça para grupos como *black* e *gay*. O símbolo do modelo não saiu na extração do texto; pela ordem do parágrafo, é um dos modelos de pesquisa.